
Rapport de Projet

formlang

Module : Théorie des langages & calculabilité

Binôme 5

ANATO K. Freddy

GUENANON K. C. Ulysse

Établissement : IUT / IFRI — Master Génie Logiciel

Enseignant : Dr. Mêton ATINDEHOU

Langage : Python 3.10+

Date : Juin 2026

Dépôt Git : https://github.com/ak4f/ANATO_GUENANON_formlang.git .

Commit final : 61bd70d (*Jour 5 : Integration, Myhill-Nerode et Pipeline complet*)

Table des matières

1	Résumé	3
2	Étage régulier & transducteurs (Jour 1)	4
2.1	Q1.1 — Expression régulière	4
2.2	Q1.2 — Du non-déterminisme à l'AFD minimal	4
2.3	Q1.3 — Facteur vs sous-séquence	5
2.4	Idempotence du FST Leetspeak	5
2.5	Miroir : one-way vs two-way	5
3	Hors-contexte (Jour 2)	7
3.1	Q2.1 — Lemme de pompage pour $D = \{[n]^n \mid n \geq 0\}$	7
3.2	Q2.2 — Grammaire des prompts bien parenthésés	7
3.3	Q2.3 — Ambiguïté et désambiguïsation	7
3.4	PDA : fonction de transition	7
4	Arbres (Jour 3) — pivot	8
4.1	Q3.1 — Définition et Déterminisme du BUTA	8
4.2	Preuve d'unité	9
4.3	Q3.2 — Échec de l'AFD de mots linéaire	9
4.4	Q3.3 — Théorème du Yield	10
4.5	Q3.4 — Produit d'intersection	10
4.6	Traces d'exécution ascendante	10
5	Calculabilité (Jour 4)	11
5.1	Q4.1 — Machine de Turing et Machine Universelle	11
5.2	Q4.2 — Calculatrice unaire	11
5.3	Q4.3 — Complexité et Indécidabilité	12
6	Intégration & Myhill–Nerode (Jour 5)	13
6.1	Q5.1 — Pipeline bout-en-bout	13
6.2	Q5.2 — Suffixes témoins et Classes d'équivalence	13
6.3	Q5.3 — Hash-consing (Bonus)	13
7	Difficultés rencontrées & choix de conception	15
8	Répartition du travail	15
	Annexe — Sortie console	16

1 Résumé

Le projet **formlang** réalise une implémentation complète de la hiérarchie de Chomsky en Python 3.10+, allant de la reconnaissance de motifs réguliers à la simulation d'un ordinateur universel. L'architecture repose sur une bibliothèque unifiée **formlang/** articulée autour de quatre applications métier :

1. **apps/morpho** — analyseur morphologique par automate d'arbres ascendant (BUTA), classifiant les structures de mots (BARE, PREFIXED, SUFFIXED, CIRCUMFIXED) ;
2. **apps/shield** — décomposeur d'attaques structurel filtrant les prompts à l'aide d'automates d'arbres réguliers et détectant le double encodage ;
3. **apps/hashcons** — table de partage de structure par Hash-consing (DAG) pour optimiser la mémoire ;
4. **apps/mtu** — simulateur de machine de Turing, machine universelle (UTM) et calculatrice unaire.

L'architecture découple rigoureusement le cœur formel (automates, grammaires, transducteurs, machines de Turing) de la logique des applications, assurant une parfaite modularité (respect de la règle *Gate*).

Résultat global des tests : 28 passed
--

2 Étage régulier & transducteurs (Jour 1)

2.1 Q1.1 — Expression régulière

Expression régulière de L_1

Le langage L_1 contient tous les mots sur $\Sigma = \{a, o, r\}$ qui possèdent le **facteur or** :

$$L_1 = (a + o + r)^* \text{ or } (a + o + r)^*$$

Justification : L_1 est le langage de tous les mots contenant le facteur consécutif **or**. L'expression $(a + o + r)^*$ capture n'importe quelle suite de lettres de Σ (y compris le mot vide ϵ). En encadrant le motif obligatoire **or**, on obtient l'ensemble exact des mots contenant ce facteur.

2.2 Q1.2 — Du non-déterminisme à l'AFD minimal

Automate Fini Non-déterministe (AFN)

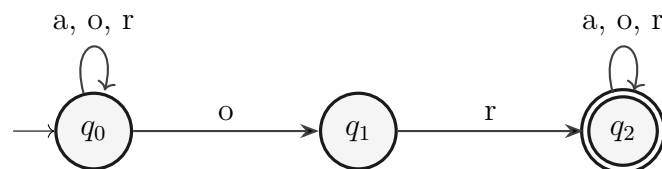


Table de transition de l'AFN :

État	a	o	r
$\rightarrow q_0$	$\{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\emptyset$	$\{q_2\}$
$* q_2$	$\{q_2\}$	$\{q_2\}$	$\{q_2\}$

Déterminisation (algorithme des sous-ensembles)

État	a	o	r
$\rightarrow S_0 = \{q_0\}$	S_0	S_1	S_0
$S_1 = \{q_0, q_1\}$	S_0	S_1	S_2
$* S_2 = \{q_0, q_2\}$	S_2	S_3	S_2
$* S_3 = \{q_0, q_1, q_2\}$	S_2	S_3	S_2

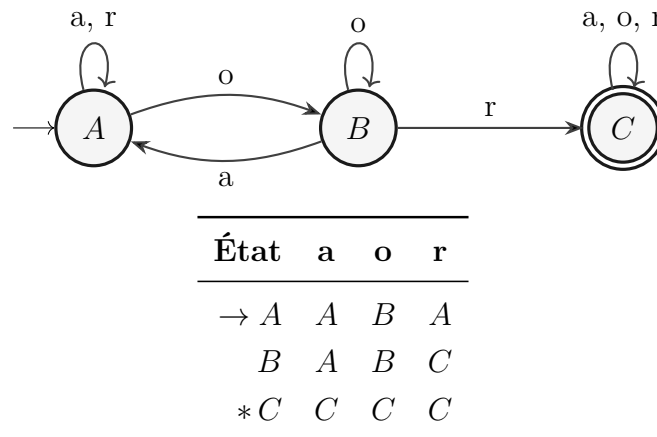
Minimisation (raffinement de Moore)

1. **Partition initiale :** $I = \{S_0, S_1\}$ (non acceptants) $II = \{S_2, S_3\}$ (acceptants).
2. **Raffinement de I :** Sur la lettre r , $S_0 \rightarrow S_0 \in I$ mais $S_1 \rightarrow S_2 \in II$. Ils sont distinguables. On sépare $I_1 = \{S_0\}$ et $I_2 = \{S_1\}$.

3. **Raffinement de II** : Sur toutes les lettres, S_2 et S_3 ont les mêmes images (II , II , II). Ils fusionnent.

L'AFD minimal possède donc **3 états** : $A = \{S_0\}$, $B = \{S_1\}$, $C = \{S_2, S_3\}$.

AFD minimal de L_1



2.3 Q1.3 — Facteur vs sous-séquence

Sous-séquence L'_1

$L'_1 = \{w \mid w \text{ contient un } o \text{ suivi, pas forcément immédiatement, d'un } r\}$

$$L'_1 = (a + o + r)^* o (a + o + r)^* r (a + o + r)^*$$

Différence : Dans L_1 , les lettres o et r forment un *facteur* (adjacentes). Dans L'_1 , elles forment une *sous-séquence* (séparées par un nombre quelconque de lettres).

Preuve de l'inclusion $L_1 \subseteq L'_1$: Soit $w \in L_1$. Il existe $u, v \in \Sigma^*$ tels que $w = u \cdot o \cdot r \cdot v = u \cdot o \cdot \epsilon \cdot r \cdot v$. Comme $\epsilon \in \Sigma^*$, w satisfait le motif de L'_1 .

Contre-exemple : Le mot oar appartient à L'_1 (le o précède le r) mais pas à L_1 (pas de facteur or consécutif).

2.4 Idempotence du FST Leetspeak

Soit T la fonction de transduction. Le mot transduit $T(w)$ ne contient plus de chiffre. Pour tout caractère alphabétique, T agit comme l'identité. Par conséquent :

$$T(T(w)) = T(w)$$

L'opération est **idempotente**.

2.5 Miroir : one-way vs two-way

Le renversement $w \mapsto w^R$ est impossible pour un transducteur *one-way* : il devrait mémoriser tout le mot dans ses états pour écrire la dernière lettre en premier, ce qui

nécessiterait un nombre infini d'états. Un transducteur *two-way* peut avancer jusqu'à la fin du ruban puis rebrousser chemin en émettant les symboles de droite à gauche.

3 Hors-contexte (Jour 2)

3.1 Q2.1 — Lemme de pompage pour $D = \{[n]^n \mid n \geq 0\}$

Supposons par l'absurde que D soit régulier. Il existe une constante de pompage $p \geq 1$. Considérons $w = [p]^p \in D$. Comme $|w| = 2p \geq p$, on décompose $w = xyz$ avec $|xy| \leq p$, $|y| \geq 1$.

Puisque $|xy| \leq p$, le facteur y ne contient que des crochets ouvrants : $y = [^k$ avec $k \geq 1$.

Pour $i = 2$, le mot pompé est $xy^2z = [^{p+k}]^p$. Or $p + k \neq p$ car $k \geq 1$, donc $xy^2z \notin D$.

Contradiction.

D n'est pas régulier : un AFD ne peut pas compter les délimiteurs. Il faut une **pile** (mémoire infinie).

3.2 Q2.2 — Grammaire des prompts bien parenthésés

Grammaire algébrique $G = (\mathcal{V}, \Sigma, R, S)$ avec $\Sigma = \{a, o, r, [,], (,)\}$:

$$S \rightarrow SS \mid [S] \mid (S) \mid a \mid o \mid r \mid \epsilon$$

3.3 Q2.3 — Ambiguïté et désambiguïisation

La grammaire est **ambiguë** car $S \rightarrow SS$ permet deux associations pour aaa :

— Association à gauche : $S \rightarrow SS \rightarrow (SS)S \rightarrow (aS)S \rightarrow (aa)a$

— Association à droite : $S \rightarrow SS \rightarrow S(SS) \rightarrow a(aS) \rightarrow a(aa)$

Grammaire **désambiguïsée** (dérivation déterministe de gauche à droite) :

$$S \rightarrow [S]S \mid (S)S \mid aS \mid oS \mid rS \mid \epsilon$$

3.4 PDA : fonction de transition

Automate à pile M

$M = (\{q_0\}, \Sigma, \Gamma, \delta, q_0, \emptyset)$ — acceptation par **pile vide**

— $\Sigma = \{a, o, r, e, [,], (,)\}$ (alphabet d'entrée)

— $\Gamma = \{[, (\}$ (alphabet de pile)

Fonction de transition δ :

1. **Empilement** : $\delta(q_0, [, \epsilon) = \{(q_0, [)\}$ et $\delta(q_0, (, \epsilon) = \{(q_0, ()\}$
2. **Dépilement** : $\delta(q_0,], [) = \{(q_0, \epsilon)\}$ et $\delta(q_0,), () = \{(q_0, \epsilon)\}$
3. **Ignorance** : $\delta(q_0, c, \epsilon) = \{(q_0, \epsilon)\}$ pour tout $c \in \{a, o, r, e\}$
4. **Rejet** : toute autre transition non définie bloque la machine.

4 Arbres (Jour 3) — pivot

4.1 Q3.1 — Définition et Déterminisme du BUTA

BUTA — Bottom-Up Tree Automaton

Un BUTA est un quadruplet $A = (Q, \Sigma, Q_f, \Delta)$ où :

- Q : ensemble fini d'états
- Σ : alphabet classé (chaque symbole a une arité)
- $Q_f \subseteq Q$: états finaux (acceptants)
- Δ : règles de transition $f(q_1, \dots, q_k) \rightarrow q$

Le BUTA est **déterministe** si pour chaque $(f, q_1, \dots, q_k)$, il existe **au plus un** état q image.

Implémentation : Δ est un dict Python associant (`symbol`, `tuple(child_states)`) à un unique état. Le parcours est **post-ordre** (feuilles $\rightarrow$ racine), coût $O(n)$.

Règles Δ de `morpho_automaton` :

Transition	$\rightarrow$	État
<code>prefix()</code>	$\rightarrow$	PRE
<code>root()</code>	$\rightarrow$	ROOT
<code>suffix()</code>	$\rightarrow$	SUF
<code>nil()</code>	$\rightarrow$	NIL
<code>prefixes(PRE, NIL)</code>	$\rightarrow$	PCHAIN
<code>prefixes(PRE, PCHAIN)</code>	$\rightarrow$	PCHAIN
<code>suffixes(SUF, NIL)</code>	$\rightarrow$	SCHAIN
<code>suffixes(SUF, SCHAIN)</code>	$\rightarrow$	SCHAIN
<code>rest(ROOT, NIL)</code>	$\rightarrow$	REST
<code>rest(ROOT, SCHAIN)</code>	$\rightarrow$	REST
<code>word(NIL, REST)</code>	$\rightarrow$	WORD
<code>word(PCHAIN, REST)</code>	$\rightarrow$	WORD

Règles Δ de `shield_automaton` :

Transition	→	État
txt()	→	SAFE
enc()	→	SAFE
ovr()	→	OVR
role()	→	ROLE
frame(SAFE)	→	SAFE
frame($x \neq$ SAFE)	→	DANGER
sys(SAFE)	→	SAFE
sys($x \neq$ SAFE)	→	DANGER

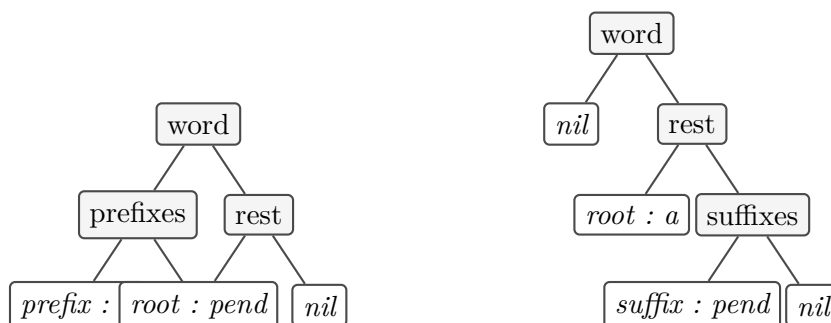
4.2 Preuve d'unité

Les deux applicationsinstancient le même moteur `TreeAutomaton` de `formlang/tree.py` :

```
# apps/morpho/automaton.py
from formlang.tree import Term, TreeAutomaton
def morpho_automaton() -> TreeAutomaton:
    A = TreeAutomaton(final_states={"WORD"})
    A.add_rule("prefix", (), "PRE")
    ...

# apps/shield/decomposer.py
from formlang.tree import Term, TreeAutomaton
def shield_automaton() -> TreeAutomaton:
    A = TreeAutomaton(final_states={"DANGER"})
    A.add_rule("txt", (), SAFE)
    ...
```

4.3 Q3.2 — Échec de l'AFD de mots linéaire



Arbre 1 (préfixe “a” + racine “pend”) Arbre 2 (racine “a” + suffixe “pend”)

Les deux ont la même frontière linéaire "a pend". Un AFD ne voit que la chaîne plate et ne peut pas distinguer la **structure syntaxique**.

4.4 Q3.3 — Théorème du Yield

Le yield d'un arbre morphologique valide est de la forme $p^* r s^*$ (préfixes, racine, suffixes), soit le langage régulier $\Sigma_{\text{pref}}^* \cdot \Sigma_{\text{root}} \cdot \Sigma_{\text{suff}}^*$. Le théorème du yield énonce que la projection linéaire des arbres réguliers est un langage **hors-contexte**.

4.5 Q3.4 — Produit d'intersection

Le produit $A_1 \times A_2$ a pour transitions :

$$f((q_1, p_1), \dots, (q_k, p_k)) \rightarrow (r_1, r_2) \quad \text{ssi} \quad f(q_1, \dots, q_k) \rightarrow r_1 \in \Delta_1 \text{ et } f(p_1, \dots, p_k) \rightarrow r_2 \in \Delta_2$$

avec états finaux $Q_{f1} \times Q_{f2}$. Alors $L(A_1 \times A_2) = L(A_1) \cap L(A_2)$.

4.6 Traces d'exécution ascendante

Trace 1 — Mot nu (BARE)

`word(nil, rest(root("livre"), nil)) :`

Feuille `nil` → NIL

Feuille `root` → ROOT

Feuille `nil` → NIL

Noeud `rest(ROOT, NIL)` → REST

Noeud `word(NIL, REST)` → **WORD** ✓

Trace 2 — Shield BLOQUÉ

`sys(role()) :`

Feuille `role` → ROLE

Noeud `sys(ROLE)` → **DANGER** ✓

5 Calculabilité (Jour 4)

5.1 Q4.1 — Machine de Turing et Machine Universelle

Encodage $\langle M \rangle$: La machine M est sérialisée en JSON injectif :

```
{
  "transitions": {
    "q0,1": ["q0", "1", "R"], ...
  },
  "start": "q0",
  "accept": ["qf"],
  "reject": []
}
```

Les clés sont triées (`sort_keys=True`), garantissant l'injectivité. L'UTM U décode et simule fidèlement :

$$U(\langle M \rangle, w) = M(w)$$

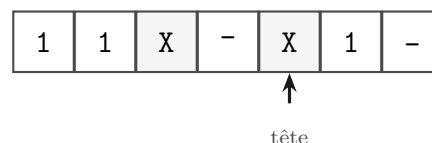
5.2 Q4.2 — Calculatrice unaire

Table ADD — expliquée ligne par ligne

Transition	Action	Explication
$(q_0, 1) \rightarrow (q_0, 1, R)$	Droite	Traverse l'opérande n .
$(q_0, +) \rightarrow (q_1, 1, R)$	Écrit 1	Remplace + par un 1 (fusion).
$(q_1, 1) \rightarrow (q_1, 1, R)$	Droite	Traverse l'opérande m .
$(q_1, _) \rightarrow (q_2, _, L)$	Gauche	Atteint la fin, recule.
$(q_2, 1) \rightarrow (q_f, _, S)$	Efface	Retire le 1 excédentaire $\rightarrow n + m$.

Table SUB — expliquée ligne par ligne

Principe : appairer les 1 de m avec ceux de n en les marquant X.



Transition	Action	Explication
$(q_0, 1) \rightarrow (q_0, 1, R)$	Droite	Traverse n vers la droite.
$(q_0, -) \rightarrow (q_1, -, R)$	Droite	Passe le séparateur.
$(q_1, X) \rightarrow (q_1, X, R)$	Droite	Saute les X déjà marqués.
$(q_1, 1) \rightarrow (q_2, X, L)$	Marque	Marque un 1 de m en X, retour gauche.
$(q_2, 1) \rightarrow (q_0, X, R)$	Marque	Marque un 1 de n en X, retour droite.
$(q_1, _) \rightarrow (q_3, _, L)$	Nettoyage	Plus de 1 dans m : nettoyer.
$(q_2, _) \rightarrow (q_4, _, R)$	Effacement	Plus de 1 dans n : résultat = 0.

MUL/DIV par composition : La multiplication est une addition répétée (n fois m).
La division est une soustraction répétée ($n \div m$) comptant les itérations.

5.3 Q4.3 — Complexité et Indécidabilité

Surcoût de simulation (sourcé) : La simulation d'une machine à k rubans sur 1 ruban est en $O(t^2)$. Sur 2 rubans, elle est en $O(t \log t)$ (*Hennie & Stearns, J. ACM, 1966*).

Indécidabilité de l'arrêt : Supposons H décidant l'arrêt. Construisons D : sur entrée $\langle M \rangle$, D boucle si H accepte, s'arrête si H rejette. $D(\langle D \rangle)$ mène à une **contradiction** dans les deux cas. Donc H n'existe pas.

6 Intégration & Myhill–Nerode (Jour 5)

6.1 Q5.1 — Pipeline bout-en-bout

Le pipeline enchaîne les trois étages de la hiérarchie sur une même entrée :

```
# python pipeline.py --word 4or
# 1. Normalisation FST : "4or" -> "aor"
# 2. AFD minimal : accepte "aor" (contient "or") -> True
# 3. PDA : pile vide a la fin -> True
```

Sortie console — Mode Word

```
normalisé(FST) : aor
facteur_or(AFD) : True
délimiteurs_ok(PDA) : True
```

Sortie console — Mode Morpho

```
{'classe(BUTA)': 'PREFIXED'}
```

Sortie console — Mode Shield

```
OK    seq(txt,txt)
OK    role (isolé)
BLOQUÉ sys(role)
BLOQUÉ seq(frame(ovr),txt)
BLOQUÉ sys(seq(txt,frame(role)))
```

6.2 Q5.2 — Suffixes témoins et Classes d'équivalence

Pour L_1 et les suffixes témoins $S = \{\epsilon, r, or, a\}$, le calcul des signatures produit exactement **3 classes d'équivalence** :

Classe	Signature	Description
A	(F, F, T, F)	Ne contient pas or , ne finit pas par o
B	(F, T, T, F)	Ne contient pas or , finit par o
C	(T, T, T, T)	Contient déjà or

Il y a **bijection** entre ces classes et les 3 états de l'AFD minimal (A, B, C).

6.3 Q5.3 — Hash-consing (Bonus)

Mesure sur **10 800 mots** ($\text{corpus_A}(600) \cup \text{corpus_B}(600)$) :

Métrique	Valeur
Total des noeuds visités	70 200
Noeuds uniques en mémoire (DAG)	16 233
Taux de compression	76,88 %

7 Difficultés rencontrées & choix de conception

1. **Ruban bi-infini de la Machine de Turing** : Nous avons utilisé un `dict[int, str]` au lieu d'un tableau de taille fixe. Cela permet les indices négatifs, l'extension dynamique et des accès en $O(1)$.
2. **Explosion combinatoire du générateur CFG** : La règle $S \rightarrow SS$ génère une infinité de dérivations. Nous avons implémenté un double élagage (nombre de terminaux et de non-terminaux) combiné à un parcours BFS avec dérivation la plus à gauche.
3. **Généricité du BUTA** : Le moteur `tree.py` stocke Δ sans hypothèse sur le type des états (chaînes ou entiers), permettant l'intégration de morpho et shield sans altérer le code de base. Respect strict de la règle *Gate*.

8 Répartition du travail

Le travail a été réalisé en binôme par **programmation en paire** (pair programming) :

Membre	Responsabilités principales
ANATO K. Freddy	Conception des algorithmes, justifications théoriques
GUENANON K. C. Ulysse	Implémentation du code, tests unitaires
<i>Revue croisée et rédaction du rapport en commun</i>	

Annexe — Sortie console

Tests unitaires (pytest -q)

```
PS> python -m pytest -q
..... [100%]
28 passed in 0.48s
```

Pipeline — Mode Word

```
PS> python -m pipeline --word "4or"
    normalise(FST) : aor
    facteur_or(AFD) : True
    delimitteurs_ok(PDA) : True
```

Pipeline — Mode Morpho

```
PS> python -m pipeline --morpho "mufafak"
{'classe(BUTA)': 'PREFIXED'}
```

Pipeline — Mode Shield

```
PS> python -m pipeline
== demo Shield (AttackDecomposer) ==
OK      seq(txt,txt)
OK      role (isole)
BLOQUE  sys(role)
BLOQUE  seq(frame(ovr),txt)
BLOQUE  sys(seq(txt,frame(role)))
```